

Part XII

Introduction to Neural Networks

Lecture Outline

- ➊ Introduction and Motivation
- ➋ Multilayer Perceptron (MLPs): The Architecture
- ➌ Expressive Power of MLPs
- ➍ Training Neural Networks

Recap: Linear Models and Their Limitations

- Previously, we discussed linear regression and classification:

$$o = f(w^\top \phi(x))$$

with

- $f(a) = a$ for regression (predicted output $\hat{y} = o$).
 - $f(a) = \text{sign}(a)$ for classification (predicted output $\hat{y} = z$).
 - $f(a) = \sigma(a)$ for classification (probability $P(1 | x, w) = z$).
- Limitations:**
 - Depend on pre-engineered features to make the data linear (Regression) or linearly separable (Classification)..
- Motivation:** Can we learn the nonlinear feature representations directly from data?

Recap: Linear Models and Their Limitations

- Previously, we discussed linear regression and classification:

$$o = f(w^\top \phi(x))$$

with

- $f(a) = a$ for regression (predicted output $\hat{y} = o$).
 - $f(a) = \text{sign}(a)$ for classification (predicted output $\hat{y} = z$).
 - $f(a) = \sigma(a)$ for classification (probability $P(1 | x, w) = z$).
- Limitations:**
 - Depend on pre-engineered features to make the data linear (Regression) or linearly separable (Classification)..
- Motivation:** Can we learn the nonlinear feature representations directly from data?
Answer: Yes! Today, we will do it with Neural Networks or Multi-Layer Perceptrons (MLPs).

Anatomy of a Neuron and MLP Architecture

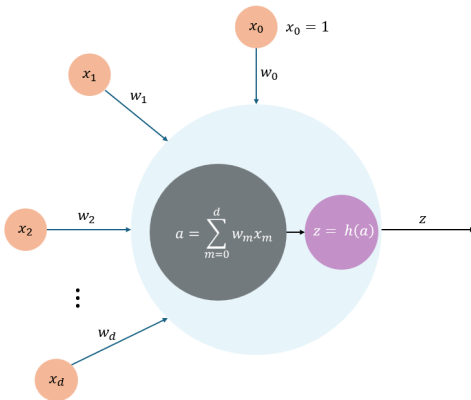
- **Idea:** Compose many simple nonlinear units (neurons/perceptrons) to capture complex functions.

Anatomy of a Neuron and MLP Architecture

- **Idea:** Compose many simple nonlinear units (neurons/perceptrons) to capture complex functions.
- For inputs $x_1, \dots, x_d$, Each neuron computes:

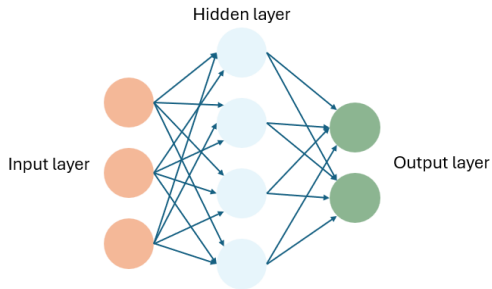
$$a = \sum_{m=0}^d w_m x_m \text{ (pre-activation)} \quad (1)$$

$$z = h(a) \text{ (activation), } h : \text{activation function (e.g., sigmoid)}. \quad (2)$$



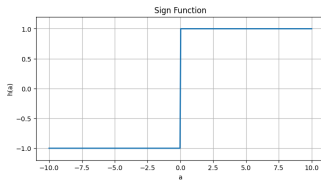
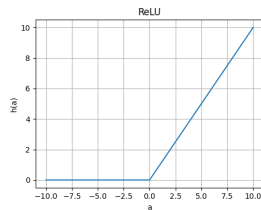
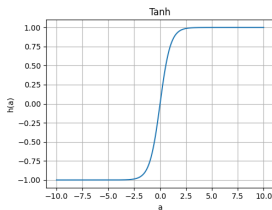
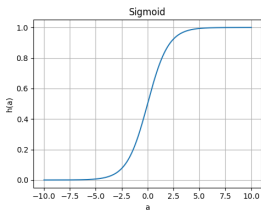
Anatomy of a Neuron and MLP Architecture

- **Idea:** Compose many simple nonlinear units (neurons/perceptrons) to capture complex functions.
- By stacking the neurons into layers, we get the *Multilayer Perceptrons (MLPs)*.



Anatomy of a Neuron and MLP Architecture

- Common Activation functions:



- For the output layer, if the task is regression, then we choose $h(a) = \text{Id}(a) = a$ (identity activation). What should we choose if the task is classification?

Expressive Power of MLPs

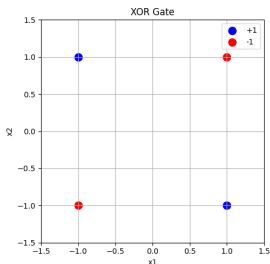
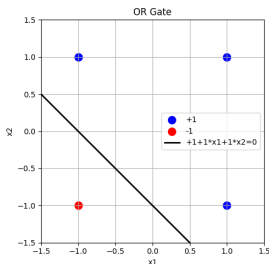
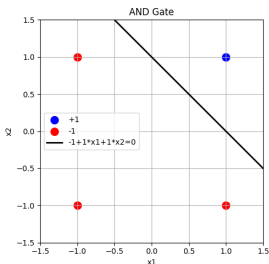
- Simplified setup:
 - Boolean Inputs (-1 or 1).
 - Boolean outputs (-1 or 1).
- **Question:** Can MLPs approximate any logic formula?
- **Question:** Can one perceptron do it? (sign function as activation).

Expressive Power of MLPs

- Simplified setup:
 - Boolean Inputs (-1 or 1).
 - Boolean outputs (-1 or 1).
- **Question:** Can MLPs approximate any logic formula?
- **Question:** Can one perceptron do it? (sign function as activation).
- One perceptron: **No**. It can only work with linearly separable functions (XOR is not linearly separable).
- MLPs: **Yes**. We can represent logic formulas with and/or/not logic gates.

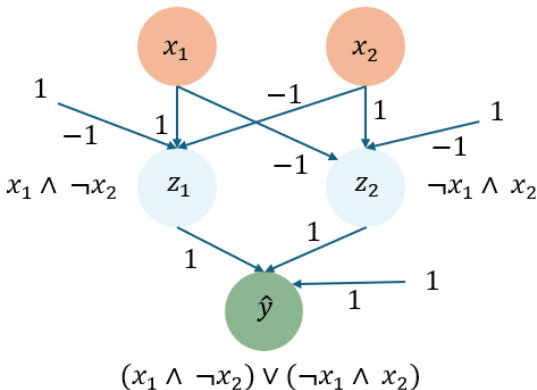
Expressive Power of MLPs

- Simplified setup:
 - Boolean Inputs (-1 or 1).
 - Boolean outputs (-1 or 1).
- **Question:** Can MLPs approximate any logic formula?
- **Question:** Can one perceptron do it? (sign function as activation).
- One perceptron: **No**. It can only work with linearly separable functions (XOR is not linearly separable).
- MLPs: **Yes**. We can represent logic formulas with and/or/not logic gates.



Expressive Power of MLPs

- Simplified setup:
 - Boolean Inputs (-1 or 1).
 - Boolean outputs (-1 or 1).
- **Question:** Can MLPs approximate any logic formula?
- **Question:** Can one perceptron do it? (sign function as activation).
- One perceptron: **No**. It can only work with linearly separable functions (XOR is not linearly separable).
- MLPs: **Yes**. We can represent logic formulas with and/or/not logic gates.



Expressive Power of MLPs

- Even more generally, the MLP is a "*Universal Approximator*".
- **Universal Approximation Theory (loosly speaking):** any continuous function $f : [0, 1]^d \rightarrow [0, 1]$ can be approximated arbitrary well by at least **one** hidden layer MLP with finite number of weights.

Expressive Power of MLPs

- **Question:** If we can approximate functions arbitrary well with one hidden layer MLP then why do we need several hidden layers (depth)?

Expressive Power of MLPs

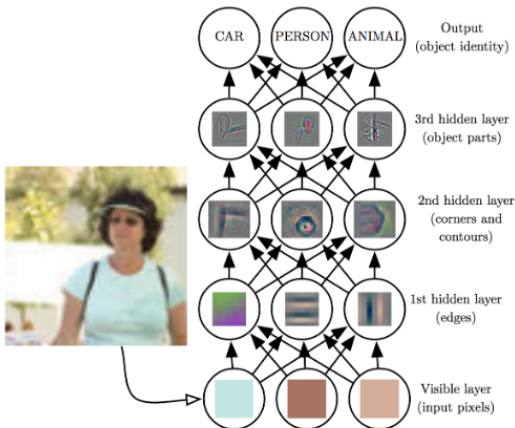
- **Question:** If we can approximate functions arbitrary well with one hidden layer MLP then why do we need several hidden layers (depth)?
- Deep nets can represent some functions more *compactly* than shallow ones.
- Example: consider the parity function

$$f_p(x_1, \dots, x_d) = \begin{cases} 1, & \sum_{m=1}^d x_m \text{ is odd.} \\ 0, & \text{otherwise.} \end{cases}$$

The function requires an *exponentially* large shallow network to be represented by an MLP. On the other hand, it can be represented by a deep network whose depth is *linear* in the number of inputs.

Expressive Power of MLPs

- Neurons will learn the features from data such that they are linearly separable toward the final layers.
- Depth can help with multiple outputs such that several outputs can share the same features in some hidden layers.



Training Neural Networks

- For N inputs $x_1, \dots, x_N \in \mathbb{R}^d$ with corresponding N outputs $y_1, \dots, y_N \in \mathbb{R}^q$, we define a loss (cost) $L_n(w)$ for each data point depending on the output of the MLP $o(w, x_n)$ (or shortly o_n). For example:

$$L_n(w) = \frac{1}{2} \|y_n - o_n\|_2^2 \quad \text{or} \quad L_n(w) = - \sum_{i=1}^q y_{n,i} \log o_{n,i}.$$

- The goal is to optimize the weights w so that we minimize the total loss (cost)

$$L(w) = \frac{1}{N} \sum_{n=1}^N L_n(w).$$

Training Neural Networks

- For N inputs $x_1, \dots, x_N \in \mathbb{R}^d$ with corresponding N outputs $y_1, \dots, y_N \in \mathbb{R}^q$, we define a loss (cost) $L_n(w)$ for each data point depending on the output of the MLP $o(w, x_n)$ (or shortly o_n). For example:

$$L_n(w) = \frac{1}{2} \|y_n - o_n\|_2^2 \quad \text{or} \quad L_n(w) = - \sum_{i=1}^q y_{n,i} \log o_{n,i}.$$

- The goal is to optimize the weights w so that we minimize the total loss (cost)

$$L(w) = \frac{1}{N} \sum_{n=1}^N L_n(w).$$

- We cannot find the minimizer in closed-form.
- Variants of Gradient descent (and their stochastic variants) are typically used for efficiency (large models and large datasets) to update the weights:

$$w \leftarrow w - \eta \nabla_w L(w).$$

Training Neural Networks

- For N inputs $x_1, \dots, x_N \in \mathbb{R}^d$ with corresponding N outputs $y_1, \dots, y_N \in \mathbb{R}^q$, we define a loss (cost) $L_n(w)$ for each data point depending on the output of the MLP $o(w, x_n)$ (or shortly o_n). For example:

$$L_n(w) = \frac{1}{2} \|y_n - o_n\|_2^2 \quad \text{or} \quad L_n(w) = - \sum_{i=1}^q y_{n,i} \log o_{n,i}.$$

- The goal is to optimize the weights w so that we minimize the total loss (cost)

$$L(w) = \frac{1}{N} \sum_{n=1}^N L_n(w).$$

- We cannot find the minimizer in closed-form.
- Variants of Gradient descent (and their stochastic variants) are typically used for efficiency (large models and large datasets) to update the weights:

$$w \leftarrow w - \eta \nabla_w L(w).$$

- Backpropagation** is a technique for calculating the gradient $\nabla_w L(w)$ efficiently.

Backpropagation: Chain Rule Overview

- Backpropagation relies mainly on the **chain rule** for computing derivatives.

Backpropagation: Chain Rule Overview

- Backpropagation relies mainly on the **chain rule** for computing derivatives.
- Scalar case:

$$\frac{df(g(x))}{dx} = \frac{df(g(x))}{dg(x)} \cdot \frac{dg(x)}{dx}$$

- Multiple components:

$$\frac{df(g_1(x), g_2(x))}{dx} = \frac{df(g_1(x), g_2(x))}{dg_1(x)} \cdot \frac{dg_1(x)}{dx} + \frac{df(g_1(x), g_2(x))}{dg_2(x)} \cdot \frac{dg_2(x)}{dx}$$

- Some notation: for $x \in \mathbb{R}^d, y \in \mathbb{R}^q$, if $y = g(x)$ and $z = f(g(x)) = f(y)$, then

$$\frac{\partial z}{\partial x_i} = \sum_{j=1}^q \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}.$$

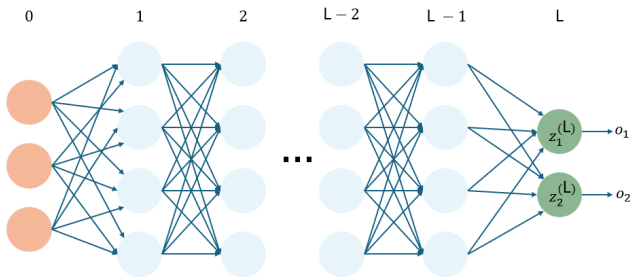
In vector notation $\nabla_x z = J_y(x)^\top \nabla_y z$ where $J_y(x)$ is the $q \times d$ jacobian matrix $(\frac{\partial y}{\partial x})$.

Setup

- In a network with L layers and q output neurons at the final layer L , we define the prediction for data point n as:

$$o_{n,j} = z_{n,j}^{(L)}, \quad j = 1, \dots, q.$$

- For clarity, we now focus on the loss for a single point L_n .
- Note each neuron i in layer ℓ can have their own activation function $h_i^{(\ell)}(\cdot)$. However, for clarity, we will write h for all of them.



- To adjust a particular weight $w_{ji}^{(\ell)}$ with $0 < \ell \leq L$ (from neuron i in layer $\ell - 1$ to neuron j in layer ℓ), we need its gradient:

$$\frac{\partial L_n}{\partial w_{ji}^{(\ell)}}.$$

- By applying the chain rule, we decompose this derivative as:

$$\frac{\partial L_n}{\partial w_{ji}^{(\ell)}} = \frac{\partial a_{n,j}^{(\ell)}}{\partial w_{ji}^{(\ell)}} \frac{\partial L_n}{\partial a_{n,j}^{(\ell)}},$$

where $a_{n,j}^{(\ell)}$ is the pre-activation of neuron j in layer ℓ for data point n .

- Recall that the pre-activation is computed as:

$$a_{n,j}^{(\ell)} = \sum_{i=1}^{I^{\ell-1}} w_{ji}^{(\ell)} z_{n,i}^{(\ell-1)},$$

with $I^{\ell-1}$ being the total number of neuros in layer ℓ .

- Thus,

$$\frac{\partial a_{n,j}^{(\ell)}}{\partial w_{ji}^{(\ell)}} = z_{n,i}^{(\ell-1)}.$$

- We define the **sensitivity** at neuron j in layer ℓ by:

$$\delta_{n,j}^{(\ell)} \stackrel{\text{def}}{=} \frac{\partial L_n}{\partial a_{n,j}^{(\ell)}}.$$

- So the gradient becomes:

$$\frac{\partial L_n}{\partial w_{ji}^{(\ell)}} = z_{n,i}^{(\ell-1)} \delta_{n,j}^{(\ell)}.$$

Output Layer Sensitivity

- In the output layer (layer L), each output neuron j (for $j = 1, \dots, q$) produces:

$$z_{n,j}^{(L)} = h(a_{n,j}^{(L)}),$$

with

$$a_{n,j}^{(L)} = \sum_{i=1}^{l^{L-1}} w_{ji}^{(L)} z_{n,i}^{(L-1)}.$$

Output Layer Sensitivity

- The loss L_n is computed based on the outputs. Using the chain rule, the sensitivity at the output becomes:

$$\delta_{n,j}^{(L)} = \frac{\partial L_n}{\partial a_{n,j}^{(L)}} = \frac{\partial z_{n,j}^{(L)}}{\partial a_{n,j}^{(L)}} \frac{\partial L_n}{\partial z_{n,j}^{(L)}} = h'(a_{n,j}^{(L)}) \frac{\partial L_n}{\partial z_{n,j}^{(L)}}.$$

Output Layer Sensitivity

- **Example: Regression**

Suppose we use mean squared error for a regression task:

$$L = \frac{1}{2} \sum_{j=1}^q (z_{n,j}^{(L)} - y_{n,j})^2 \quad (z_{n,j}^{(L)} = o_{n,j}).$$

Then,

$$\frac{\partial L}{\partial z_{n,j}^{(L)}} = z_{n,j}^{(L)} - y_{n,j}.$$

If we use the identity activation $h(a) = a$ (so $h'(a) = 1$), we obtain:

$$\delta_{n,j}^{(L)} = z_{n,j}^{(L)} - y_{n,j}.$$

Output Layer Sensitivity

- **Example: Classification**

- *Binary Classification with Sigmoid:* With a sigmoid activation $h(a) = \sigma(a) = \frac{1}{1+e^{-a}}$ and binary cross-entropy loss,

$$L_n = - \left[y_{n,j} \ln(z_{n,j}^{(L)}) + (1 - y_{n,j}) \ln(1 - z_{n,j}^{(L)}) \right].$$

It can be shown (by applying the chain rule) that:

$$\delta_{n,j}^{(L)} = z_{n,j}^{(L)} - y_{n,j}.$$

- *Multi-class Classification with Softmax:* When using softmax in the output layer:

$$z_{n,j}^{(L)} = \frac{e^{a_{n,j}^{(L)}}}{\sum_{k=1}^q e^{a_{n,k}^{(L)}}},$$

combined with the cross-entropy loss

$$L = - \sum_{j=1}^q y_{n,j} \ln(z_{n,j}^{(L)}),$$

the derivation yields (after some algebra):

$$\delta_{n,j}^{(L)} = z_{n,j}^{(L)} - y_{n,j}.$$

Hidden Layer Sensitivities: Propagating the Error

- Denoting I^ℓ to be the number of neuros in layer $1 \leq \ell \leq L$, then in any hidden layer ℓ (with $1 \leq \ell < L$), a neuron's pre-activation is given by:

$$a_{n,j}^{(\ell)} = \sum_{i=1}^{I^{(\ell-1)}} w_{ji}^{(\ell)} z_{n,i}^{(\ell-1)}.$$

The activation of layer ℓ is:

$$z_{n,j}^{(\ell)} = h(a_{n,j}^{(\ell)}).$$

Hidden Layer Sensitivities: Propagating the Error

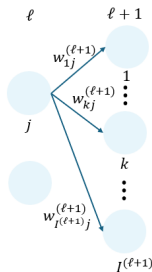
- The loss depends on $a_{n,j}^{(\ell)}$ only indirectly (via later layers). To compute the sensitivity:

$$\delta_{n,j}^{(\ell)} = \frac{\partial L_n}{\partial a_{n,j}^{(\ell)}},$$

we apply the chain rule over the neurons in layer $\ell + 1$:

$$\delta_{n,j}^{(\ell)} = \sum_{k=1}^{I^{(\ell+1)}} \frac{\partial z_{n,j}^{(\ell)}}{\partial a_{n,j}^{(\ell)}} \frac{\partial a_{n,k}^{(\ell+1)}}{\partial z_{n,j}^{(\ell)}} \frac{\partial L_n}{\partial a_{n,k}^{(\ell+1)}}.$$

Note that $\frac{\partial L_n}{\partial a_{n,k}^{(\ell+1)}} = \delta_{n,k}^{(\ell+1)}$ (the sensitivity of neuron k from the **next** layer).



Hidden Layer Sensitivities: Propagating the Error



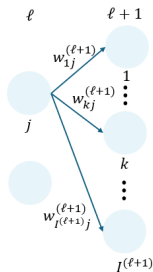
$$\delta_{n,j}^{(\ell)} = \sum_{k=1}^{I^{(\ell+1)}} \frac{\partial z_{n,j}^{(\ell)}}{\partial a_{n,j}^{(\ell)}} \frac{\partial a_{n,k}^{(\ell+1)}}{\partial z_{n,j}^{(\ell)}} \frac{\partial L_n}{\partial a_{n,k}^{(\ell+1)}}.$$

- Since $z_{n,j}^{(\ell)} = h(a_{n,j}^{(\ell)})$ and the pre-activation in layer $\ell + 1$ for neuron k is:

$$a_{n,k}^{(\ell+1)} = \sum_{j=1}^{I^{(\ell)}} w_{kj}^{(\ell+1)} z_{n,j}^{(\ell)},$$

we have:

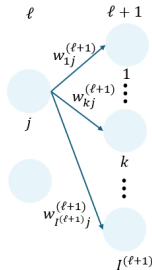
$$\frac{\partial z_{n,j}^{(\ell)}}{\partial a_{n,j}^{(\ell)}} = h'(a_{n,j}^{(\ell)}), \quad \frac{\partial a_{n,k}^{(\ell+1)}}{\partial z_{n,j}^{(\ell)}} = w_{kj}^{(\ell+1)}$$



Hidden Layer Sensitivities: Propagating the Error

- Thus, the sensitivity for a neuron in a hidden layer becomes:

$$\delta_{n,j}^{(\ell)} = h'(a_{n,j}^{(\ell)}) \sum_{k=1}^{I^{(\ell+1)}} w_{kj}^{(\ell+1)} \delta_{n,k}^{(\ell+1)}.$$



- Interpretation:** We compute the sensitivities backward! (Backpropagation). We start with the output layer sensitivities $\delta_{n,1}^L, \dots, \delta_{n,q}^L$ (assuming q outputs) and go back using the formula.

Weight Gradient and Update Rule

- Recall: We derived that for a weight $w_{ji}^{(\ell)}$,

$$\frac{\partial L_n}{\partial w_{ji}^{(\ell)}} = \delta_{n,j}^{(\ell)} z_{n,i}^{(\ell-1)}.$$

Weight Gradient and Update Rule

- Recall: We derived that for a weight $w_{ji}^{(\ell)}$,

$$\frac{\partial L_n}{\partial w_{ji}^{(\ell)}} = \delta_{n,j}^{(\ell)} z_{n,i}^{(\ell-1)}.$$

With this gradient computed for each data point n , we can then use either standard gradient descent (averaging the gradient based on the total loss) or stochastic gradient descent (averaging over mini-batches B of data points with number of points (size) $|B|$).

$$w_{ji}^{(\ell)} \leftarrow w_{ji}^{(\ell)} - \eta \frac{1}{N} \sum_{n=1}^N \delta_{n,j}^{(\ell)} z_{n,i}^{(\ell-1)}.$$

Or

$$w_{ji}^{(\ell)} \leftarrow w_{ji}^{(\ell)} - \eta \frac{1}{|B|} \sum_{n \in B} \delta_{n,j}^{(\ell)} z_{n,i}^{(\ell-1)}.$$

Summary of Backpropagation Steps

Step 1. Forward Pass: Compute for each data point n and for each layer ℓ :

$$a_{n,j}^{(\ell)} = \sum_{i=1}^{l^{(\ell-1)}} w_{ji}^{(\ell)} z_{n,i}^{(\ell-1)} \quad \text{and} \quad z_{n,j}^{(\ell)} = h(a_{n,j}^{(\ell)}).$$

Summary of Backpropagation Steps

Step 2. Output Sensitivity: For each output neuron j (with $j = 1, \dots, q$):

$$\delta_{n,j}^{(L)} = h'(a_{n,j}^{(L)}) \frac{\partial L_n}{\partial z_{n,j}^{(L)}}.$$

with $\hat{y}_{n,j} = z_{n,j}^{(L)}$.

Summary of Backpropagation Steps

Step 3. Hidden Layer Sensitivities: For each hidden layer ℓ :

$$\delta_{n,j}^{(\ell)} = h'(a_{n,j}^{(\ell)}) \sum_{k=1}^{l^{(\ell+1)}} w_{jk}^{(\ell+1)} \delta_{n,k}^{(\ell+1)}.$$

Summary of Backpropagation Steps

Step 4. Weight Gradients: For every weight $w_{ji}^{(\ell)}$:

$$\frac{\partial L_n}{\partial w_{ji}^{(\ell)}} = \delta_{n,j}^{(\ell)} z_{n,i}^{(\ell-1)}.$$

Summary of Backpropagation Steps

Step 1. Forward Pass: Compute for each data point n and for each layer ℓ :

$$a_{n,j}^{(\ell)} = \sum_{i=1}^{l^{(\ell-1)}} w_{ji}^{(\ell)} z_{n,i}^{(\ell-1)} \quad \text{and} \quad z_{n,j}^{(\ell)} = h(a_{n,j}^{(\ell)}).$$

Step 2. Output Sensitivity: For each output neuron j (with $j = 1, \dots, q$):

$$\delta_{n,j}^{(L)} = h'(a_{n,j}^{(L)}) \frac{\partial L_n}{\partial z_{n,j}^{(L)}}.$$

Step 3. Hidden Layer Sensitivities: For each hidden layer ℓ :

$$\delta_{n,j}^{(\ell)} = h'(a_{n,j}^{(\ell)}) \sum_{k=1}^{l^{(\ell+1)}} w_{jk}^{(\ell+1)} \delta_{n,k}^{(\ell+1)}.$$

Step 4. Weight Gradients: For every weight $w_{ji}^{(\ell)}$:

$$\frac{\partial L_n}{\partial w_{ji}^{(\ell)}} = \delta_{n,j}^{(\ell)} z_{n,i}^{(\ell-1)}.$$

Practical Considerations

- Other gradient-based (no hessian) optimization algorithms than SGD and gradient descent are often used. Example: Momentum, ADAM.
- Regularization is often employed.

Handling Non-Differentiable Activation Functions

- **Sign Activation:**

- The sign function outputs ± 1 but is non-differentiable at $a = 0$ and has zero gradient almost everywhere. Possible practical solutions:
 - *Smooth Approximation:* Replace the sign function with a smooth alternative (e.g., $\tanh(a)$) during training.
 - Take $h'(a) = 1$.

- **ReLU Activation:**

- Defined as $h(a) = \max(0, a)$, ReLU is non-differentiable at $a = 0$. Possible practical solutions:
 - In practice, the derivative at $a = 0$ is typically defined as 0 so

$$h'(a) = \begin{cases} 0, & a \leq 0 \\ 1, & \text{otherwise.} \end{cases}$$

Vanishing and Exploding Gradient Problem

- **Overview:** In deep networks, gradients may:
 - **Vanish:** Gradients become very small, slowing early-layer learning.
 - **Explode:** Gradients grow excessively, causing unstable updates.
- **Contributing Factors:**
 - **Activations:** Functions like sigmoid/tanh can lead to vanishing gradients (gradients can be almost zero for a large range of inputs in the saturation regions).
 - **Initialization:** Weights that are too small or large may worsen gradient flow.

Vanishing and Exploding Gradient Problem

- **Overview:** In deep networks, gradients may:
 - **Vanish:** Gradients become very small, slowing early-layer learning.
 - **Explode:** Gradients grow excessively, causing unstable updates.
- **Contributing Factors:**
 - **Activations:** Functions like sigmoid/tanh can lead to vanishing gradients (gradients can be almost zero for a large range of inputs in the saturation regions).
 - **Initialization:** Weights that are too small or large may worsen gradient flow.
- **Mitigation Techniques:**
 - Use proper initialization (e.g., Xavier, He).
 - Choose activations like ReLU instead of sigmoid (no saturation for positive inputs).
 - Apply normalization methods (Batch/Layer Norm) and gradient clipping (in case of exploding gradients).
 - Incorporate architectural designs like skip connections (e.g., residual neural networks).

- **From Linear Models to Neural Networks:**
 - Linear regression/classification requires pre-engineered features.
 - Motivation to learn nonlinear representations directly from data.
- **MLP Architecture:**
 - Composes many simple nonlinear neurons.
 - Stacking neurons into layers yields hierarchical feature learning.
- **Expressive Power:**
 - MLPs are universal approximators.
 - Deep networks can represent complex functions (e.g., parity) more compactly than shallow ones.
- **Training via Backpropagation:**
 - Forward pass: compute weighted activations.
 - Backward pass: use the chain rule to compute gradients.
 - Weight updates use gradient descent (or its variants).
- **Practical Considerations:**
 - Handle non-differentiable activations (e.g., sign, ReLU) with approximations.
 - Mitigate vanishing/exploding gradients using proper initialization, normalization, and advanced optimizers.